1. Cover Page

- **Project Title:** Automated Seed Viability Decay & Inventory Management System
- **Course Name:** Programming in Java
- **Student Name:** Shlok Mahesh Thakur
- **Registration Number:** 25BAI10020

2. INTRODUCTION:

Good crop yields depend heavily on the quality of the seeds used. When seeds sit in storage, they slowly lose their ability to sprout and grow into healthy plants. Most small seed stores and warehouses still track stock on paper or basic spreadsheets, which only count bags of seeds without checking if the seeds inside are still alive.

This project is a simple, easy-to-use Java program that tracks stored seeds, figures out their current quality based on how old they are, and makes sure dead or expired seeds are never sold to farmers.

3. PROBLEM STATEMENT:

Seed lots undergo progressive physiological deterioration from the moment of harvest. When seed stock is held over prolonged periods, manual tracking cannot accurately predict when a batch falls below regulatory planting thresholds. The lack of proactive, programmatic quality enforcement results in:

- Distribution of degraded, low-germination seed lots to growers.
- Unchecked fulfillment of orders utilizing expired seed inventory.
- Critical operational errors such as inventory overselling and stock overdrafts.

An automated management system is required to compute chronological viability decay, monitor multi-crop reserves, enforce automated threshold gating, and halt operations involving compromised stock.

4. FUNCTIONAL REQUIREMENTS:

The system implements three primary operational modules:

FR1: Seed Batch & Inventory Lifecycle Management

- Register seed batches specifying Batch ID, Crop Type (RICE, MAIZE, RAGI, SOYBEAN), quantity, harvest date, and baseline viability.
- Query individual batch records via unique identifiers using encapsulated lookup mechanisms.
- Deduct physical stock during dispatch transactions while asserting quantity availability.

FR2: Temporal Viability Evaluation Engine

- Calculate time elapsed between harvest dates and the current system date (LocalDate.now()).
- Apply crop-specific degradation formulas to derive real-time germination percentages.

FR3: Inventory Audit & Reporting Engine

- Iterate across registered batches to produce consolidated audit reports.
- Trap expired batches within error-handling blocks and report an explicit EXPIRED status without terminating the reporting routine.

5. NON-FUNCTIONAL REQUIREMENTS:

Pursuant to system reliability and production standards, the software satisfies the following criteria:

- **NFR1: Reliability:** System transactions maintain referential and quantitative integrity. The system prevents data corruption by employing

validation boundaries that reject negative deductions, null lookups, and operations on expired inventory.

- **NFR2: Error Handling Strategy:** The software utilizes robust application-specific exceptions (ExpiredSeedException, InadequateStockException) to cleanly catch and isolate runtime violations, delivering actionable feedback to users without unhandled halts.
- **NFR3: Maintainability:** The codebase adheres to strict separation of concerns through an enterprise package structure (model, service, exception), facilitating independent updates to calculation models or storage services without cross-layer friction.
- **NFR4: Usability:** The command-line interface provides clean, structured interaction workflows featuring clear prompts, explicit batch statuses, and aligned tabular outputs that minimize user operational error.

6. System Architecture

The project follows a layered architectural pattern that decouples presentation, domain models, business logic, and error management:

- **Presentation Layer (Main.java):** Shows the menu to the user and takes their choices.

Service Layer: Does all the heavy lifting:

- **InventoryService:** Stores and updates seed stock.
- **ViabilityCalculationService:** Calculates seed age and quality.
- **ReportService:** Puts the data together into a readable report.

Model Layer (SeedBatch.java, CropType.java): Holds the seed batch information and crop rules.

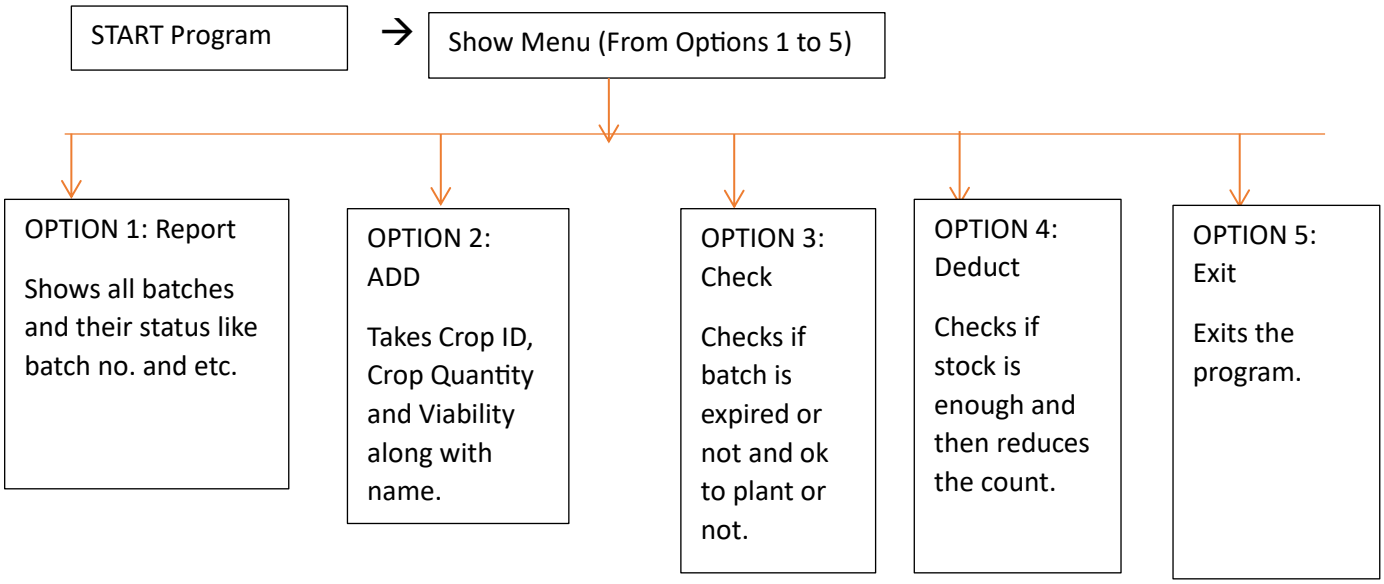
Exception Layer: Contains custom safety alerts (ExpiredSeedException, InadequateStockException).

7. Design Diagrams:

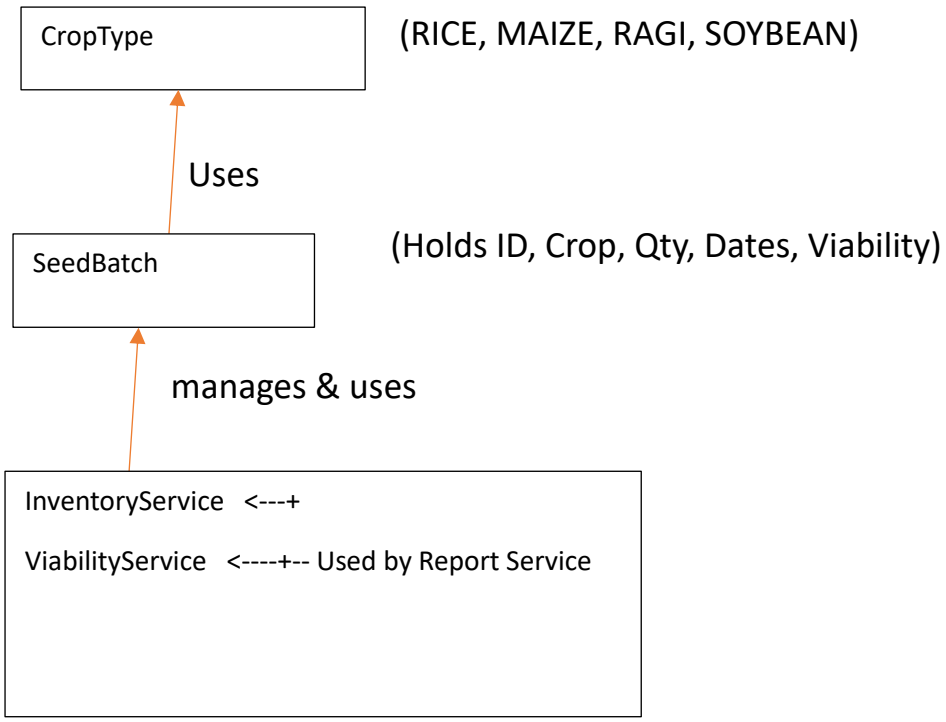
Seed Viability Management System (To be Entered by the User)

1. View Inventory & Quality Report
2. Add New Seed Batch
3. Check Single Batch Status
4. Deduct Stock When Sold

WORKFLOW DIAGRAM:



CLASS DIAGRAM:



8. Design Decisions & Rationale:

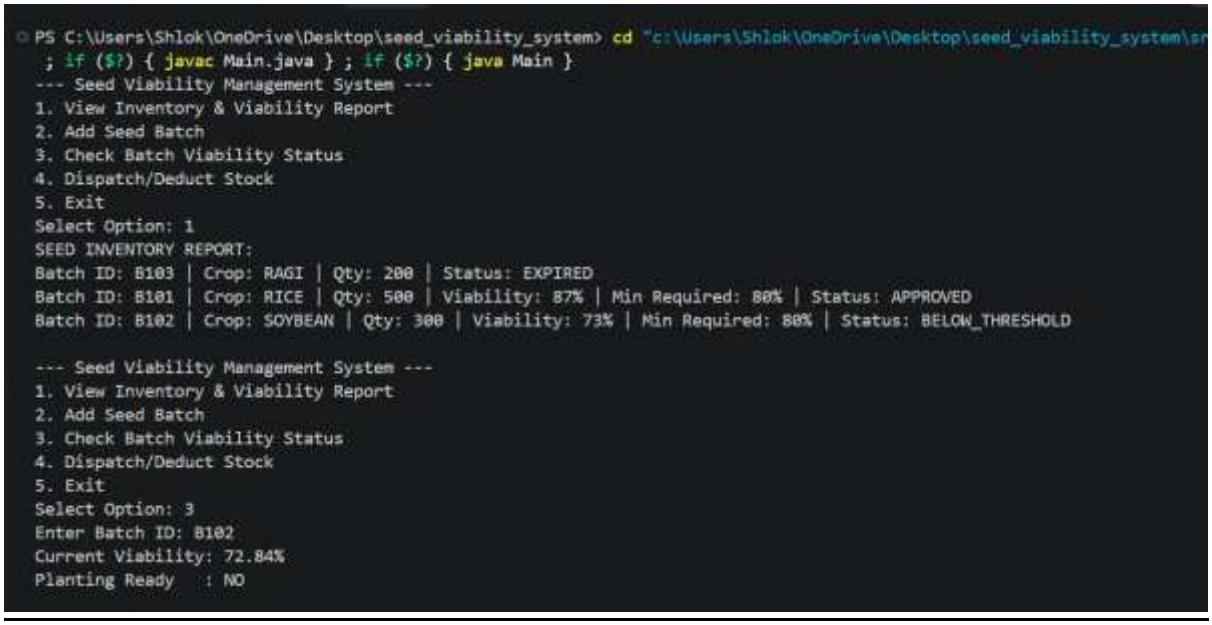
- **Enumerated Crop Definitions (CropType):** Using an enum enforces strict domain control over allowable crops (RICE, MAIZE, RAGI, SOYBEAN), eliminating arbitrary string typographical errors while co-locating standard legal germination baselines with their respective crops.
- **Modern Date/Time API (java.time.LocalDate):** Java's modern date engine replaces legacy java.util.Date, allowing immutable, thread-safe date arithmetic without calendar conversions.
- **Encapsulation via Optional<T>:** InventoryService.getBatch() returns Optional<SeedBatch>, requiring the caller to explicitly handle absent records and eliminating unhandled NullPointerExceptions.
- **Custom Domain Exceptions:** Constructing checked exceptions (ExpiredSeedException, InadequateStockException) clearly distinguishes domain logic failures from infrastructure faults.

9. Implementation Details:

The solution is organized under clean source directories:

- **model/CropType.java:** Defines the 4 permitted crops and establishes the minimum germination thresholds: Rice (80%), Maize (75%), Ragi (70%), and Soybean (75%).
- **model/SeedBatch.java:** Models physical seed attributes, tracking batch numbers, initial viability, quantities, and chronological lifespans.
- **service/InventoryService.java:** Employs an internal collection mapping to track active batches, validate stock subtractions, and manage stock deductions.
- **service/ViabilityCalculationService.java:** Applies monthly degradation coefficients to initial viability baselines while evaluating current system dates against expiry cutoffs.
- **service/ReportService.java:** Aggregates data from both services, generating an operational status audit for store operators.
- **Main.java:** Serves as the interactive CLI application, presenting menu loops, handling user inputs via Scanner, and catching exceptions.

10. OUTPUT SCREENSHOT:



11. Testing Approach:

I tested the system manually in the terminal by probing boundary conditions and screenshot of the output of the console is in the 10 point i.e. Output Screenshot:

- **Expiration Boundary:** I entered batch B103 (harvested 24 months ago with an expiry date set in the past) to check that the service catches ExpiredSeedException and tags the report row as EXPIRED instead of crashing.
- **Over-Deduction Check:** I tried deducting 600 kg from batch B101 (which only had 500 kg). The system refused the transaction and cleanly printed the error message.

12. Challenges Faced:

- **Handling Unchecked Optional Returns:** Initial versions experienced runtime reference errors during batch queries; this was resolved by implementing. Or Else(null) lookups paired with explicit verification checks.
- **Output Formatting in Console Streams:** Raw concatenation previously merged numerical values together (e.g., 300.073%80%). This was resolved by restructuring output strings with clear column dividers and labels.
- **Classpath Resolution:** Compiling from child directories created compiler symbol errors; standardizing compilation commands from the parent src directory resolved package resolution across the application.

13. Learnings & Key Takeaways:

- Practical application of layered software architecture in Java, decoupling presentation code from business rules.
- Proper design and implementation of domain-specific checked exceptions for operational safety.
- Effective use of java.time APIs for business calculations over manual time-conversion formulas.

- Git repository management, enforcing proper file tracking and documentation standards.

14. Future Enhancements:

- Persistent Database Integration: Replace in-memory data structures with an embedded relational database (e.g., SQLite or PostgreSQL via JDBC) to persist inventory across restarts.
- **Environmental Sensor Integration:** Integrate temperature and relative humidity tracking to dynamically alter decay rates based on actual storage conditions.
- **Automated Re-order Triggers:** Automatically generate supplier purchase requests when active stock falls below safety reserves.

15. References:

- Oracle Java 17 Documentation
- Standard Seed Viability & Germination Principles (ISTA Rules).